Comparison of abmneo and ABM - abmneo part

Sasha D. Hafner

08 May, 2026 10:08

Overview

This demo is meant to compare and align abmneo with the ABM package.

```
devtools::load_all()
```

```
## i Loading abmneo
```

Set pars

Original ABM values from `ABM_fit/functions/setInitPars.R` at 1f98ed2590948562c24df9ac7272bf9c85c82f34.

```
mstoich <- matrix(
  c(
    -1, -1, -1, -1, -1, -1, -1,
     0,  0,  0,  0,  0,  0, -0.50156,
     1,  1,  1,  1,  1,  1,  0
  ),
  nrow = 3,
  byrow = TRUE,
  dimnames = list(
    c('VFA', 'SO4', 'CH4'),
    c('m0', 'm1', 'm2', 'm3', 'm4', 'm5', 'sr1')
  )
)

mstoich
```

```
##     m0 m1 m2 m3 m4 m5      sr1
## VFA -1 -1 -1 -1 -1 -1 -1.00000
## SO4  0  0  0  0  0  0 -0.50156
## CH4  1  1  1  1  1  1  0.00000
```

```
# For ksmat, SO4 value is from mic.pars2.0 in ABM repo
ksmat <- matrix(
  c(1.15, 1.15, 1.15, 1.15, 1.15, 1.15, 0.46,
      NA,   NA,   NA,   NA,   NA,   NA, 0.00694),
  nrow = 2,
  byrow = TRUE,
  dimnames = list(
    c('VFA', 'SO4'),
    c('m0', 'm1', 'm2', 'm3', 'm4', 'm5', 'sr1')
  )
)
```

```
ksmat
```

```
##       m0   m1   m2   m3   m4   m5     sr1
## VFA 1.15 1.15 1.15 1.15 1.15 1.15 0.46000
## SO4   NA   NA   NA   NA   NA   NA 0.00694
```

```
grp_pars <- list(
  grps = c('m0', 'm1', 'm2','m3','m4','m5','sr1'),
  yield = c(default = 0.05, sr1 = 0.065),
  xa_fresh = c(m0 = 0.06, m1 = 0.06, m2 = 0.06, m3 = 0.06, m4 = 0.06, m5 = 0.06, sr1 = 0.06),
  xa_init = c(default = 0.1),
  d_max = c(default = 0.02),
  qhat_opt =  c(m0 = 0.46289, m1 = 0.74568, m2 = 0.9006605, m3 = 2.79672, m4 = 4.66012, m5 = 7.4601, sr
  T_opt = c(m0 = 18, m1 = 18, m2 = 28, m3 = 36, m4 = 43.75, m5 = 55, sr1 = 43.75),
  T_min = c(m0 = 0, m1 = 9.67, m2 = 9.67, m3 = 15, m4 = 26.25, m5 = 30, sr1 = 0),
  T_max = c(m0 = 25, m1 = 25, m2 = 38, m3 = 45, m4 = 51.25, m5 = 60, sr1 = 51.25),
  ksmat = ksmat,
  mstoich = mstoich
)

grp_pars
```

```
## $grps
## [1] "m0"  "m1"  "m2"  "m3"  "m4"  "m5"  "sr1"
##
## $yield
## default     sr1
##   0.050   0.065
##
## $xa_fresh
##   m0   m1   m2   m3   m4   m5  sr1
## 0.06 0.06 0.06 0.06 0.06 0.06 0.06
##
## $xa_init
## default
##     0.1
##
## $d_max
## default
##    0.02
##
## $qhat_opt
##        m0        m1        m2        m3        m4        m5       sr1
## 0.4628900 0.7456800 0.9006605 2.7967200 4.6601200 7.4601000 0.0000000
##
## $T_opt
##    m0    m1    m2    m3    m4    m5   sr1
## 18.00 18.00 28.00 36.00 43.75 55.00 43.75
##
## $T_min
##    m0    m1    m2    m3    m4    m5   sr1
##  0.00  9.67  9.67 15.00 26.25 30.00  0.00
##
## $T_max
##    m0    m1    m2    m3    m4    m5   sr1
```

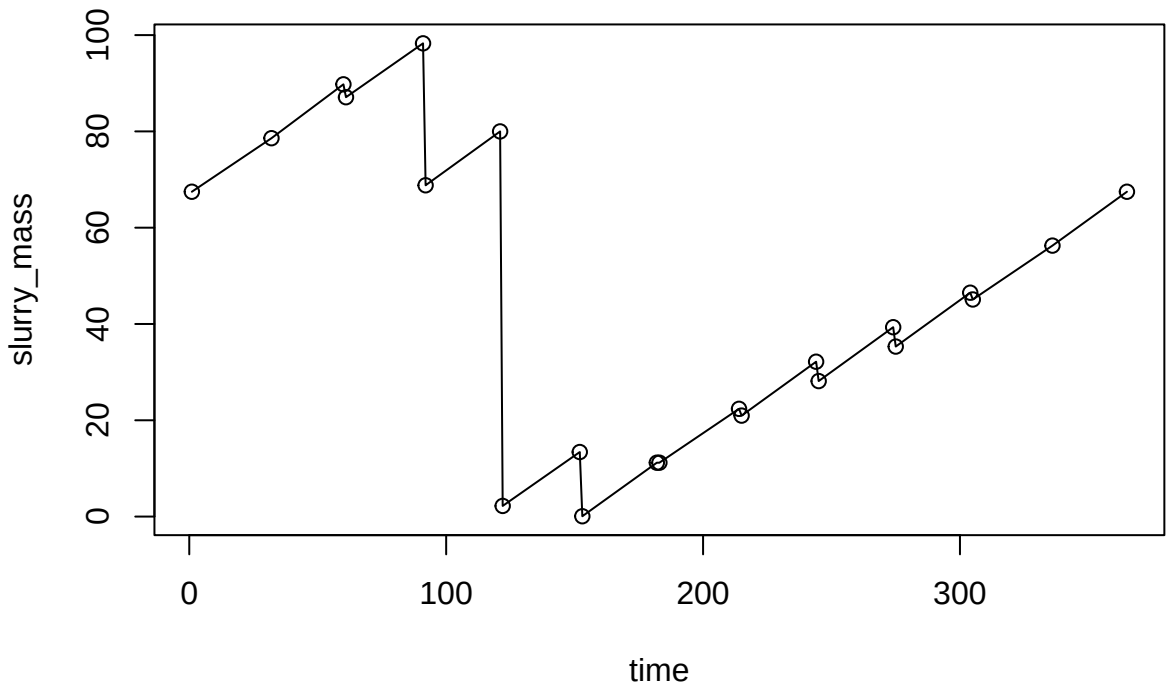
```
## 25.00 25.00 38.00 45.00 51.25 60.00 51.25
##
## $ksmat
##       m0   m1   m2   m3   m4   m5     sr1
## VFA 1.15 1.15 1.15 1.15 1.15 1.15 0.46000
## SO4   NA   NA   NA   NA   NA   NA 0.00694
##
## $mstoich
##     m0 m1 m2 m3 m4 m5      sr1
## VFA -1 -1 -1 -1 -1 -1 -1.00000
## SO4  0  0  0  0  0  0 -0.50156
## CH4  1  1  1  1  1  1  0.00000
```

Influent pars from man_pars2.0.

```
inf_pars <- list(
  VFA_fresh = 1.7,
  VFA_init = 1.7,
  comps = c('SO4'),
  comp_fresh = c(SO4 = 0.2),
  comp_init = c(SO4 = 0.2),
  pH = 7,
  dens = 1000
)
```

Slurry level and temperature, outdoor pig slurry tank

```
mass_series <- read.csv('level_pig_od_ref.csv')
plot(slurry_mass ~ time, data = mass_series, type = 'o')
```

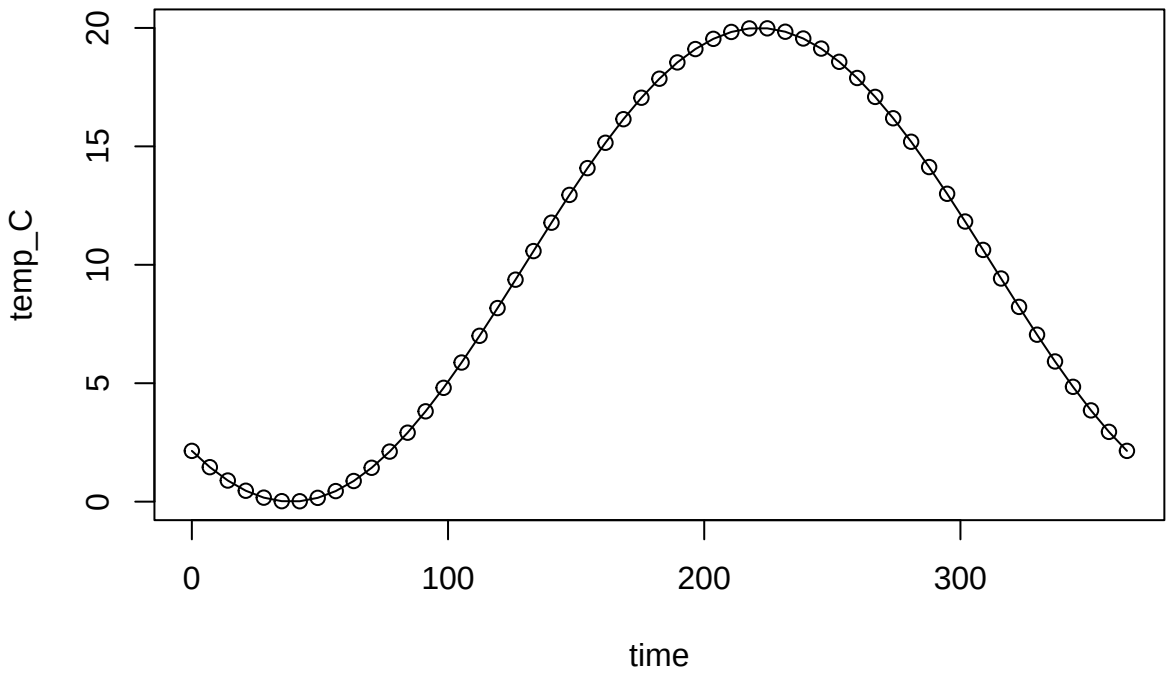



```
stor_ser <- list(
  type = 'series',
  dat = mass_series,
  storage_depth = 2,
  area = 0.03,
  temp_C = 0,
  resid_enrich = 0.9
)
```

```
temp_dat <- data.frame(time = seq(0, 365, length.out = 53))

temp_dat$temp_C <- 10 + 10 * sin((temp_dat$time - 130) / 365 * 2 * pi)

plot(temp_C ~ time, data = temp_dat, type = 'o')
```



Substrates and hydrolysis

```
sub_pars <- list(
  subs = c('starch', 'Cfat', 'CPs', 'CPf', 'RFd'),
  sub_enrich = c('starch', 'Cfat', 'CPs', 'RFd'),
  xd = 'xd',
  arrA = c(starch = 0.557 * 5 * 3.61e12, Cfat = 0.1 * 0.557 * 3.61e12, CPs = 0.557 * 2.065 * 3.61e12, CI
  arrE = c(default = 8.16e4),
  sub_fresh = c(starch = 5.25, Cfat = 27.6, CPs = 21.1 * 0.55, CPf = 21.1 * 0.45, RFd = 25.4, xd = 0),
  sub_init = c(starch = 5.25, Cfat = 27.6, CPs = 21.1 * 0.55, CPf = 21.1 * 0.45, RFd = 25.4, xd = 0)
)
```

Chem pars. COD_conv is g COD per g whatever (here g CH4-C and g SO4-S). Needed for COD balance. Of course VFA and substrates are all in COD units. And O2 is oxygen.

```
chem_pars <- list(COD_conv = c(CH4 = 5.32, SO4 = -1.9938), gases = c('CH4'))
```

To get COD of SO4-2:

`micstoich::micstoich('CH3COOH', acceptor = 'SO4-2', product = 'H2S')` How much COD consumed?
`micstoich::calcCOD('CH3COOH', per = 'mol') * 0.125` How much SO4-S consumed? `micstoich::molmass('SO4-2', 'S') * 0.125` 4.0125 / 8 8 / 4.0125

Try to run

Q

```
devtools::load_all()
```

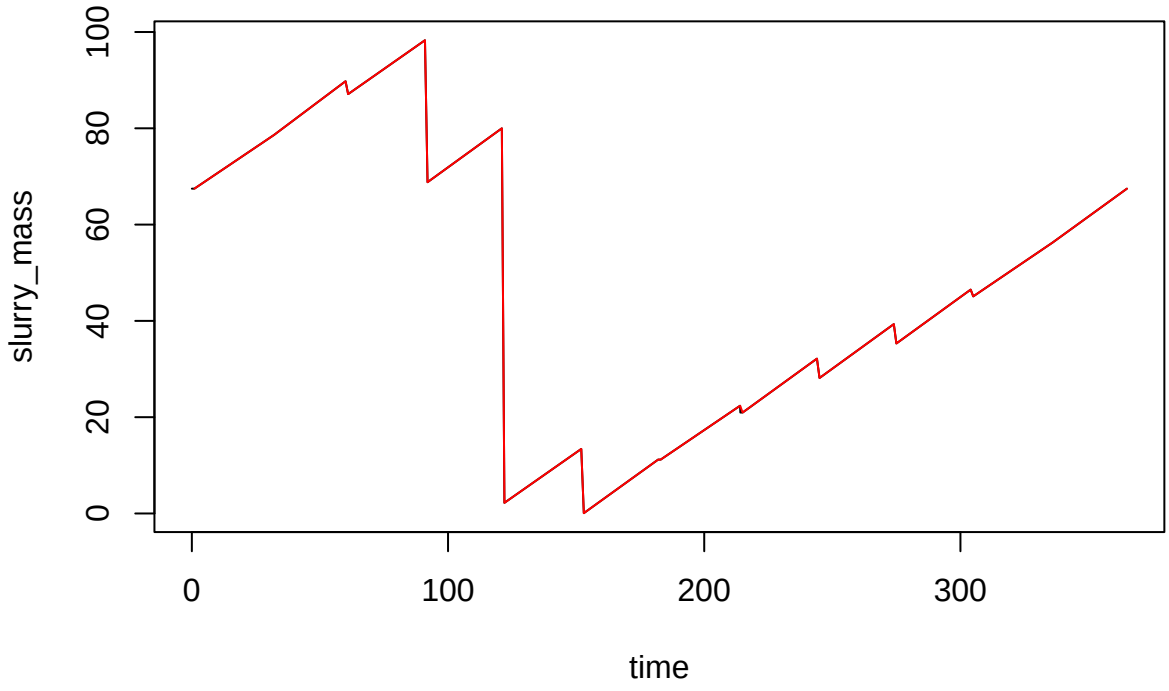
```
## i Loading abmneo
```

```
system.time(
  neo_out <- abmneo(
    storage = stor_ser,
    365,
    delta_t = 1,
    inf_pars = inf_pars,
    grp_pars = grp_pars,
    sub_pars = sub_pars,
    chem_pars = chem_pars,
    var_pars = list(var = temp_dat),
    startup = 2
  )
)
```

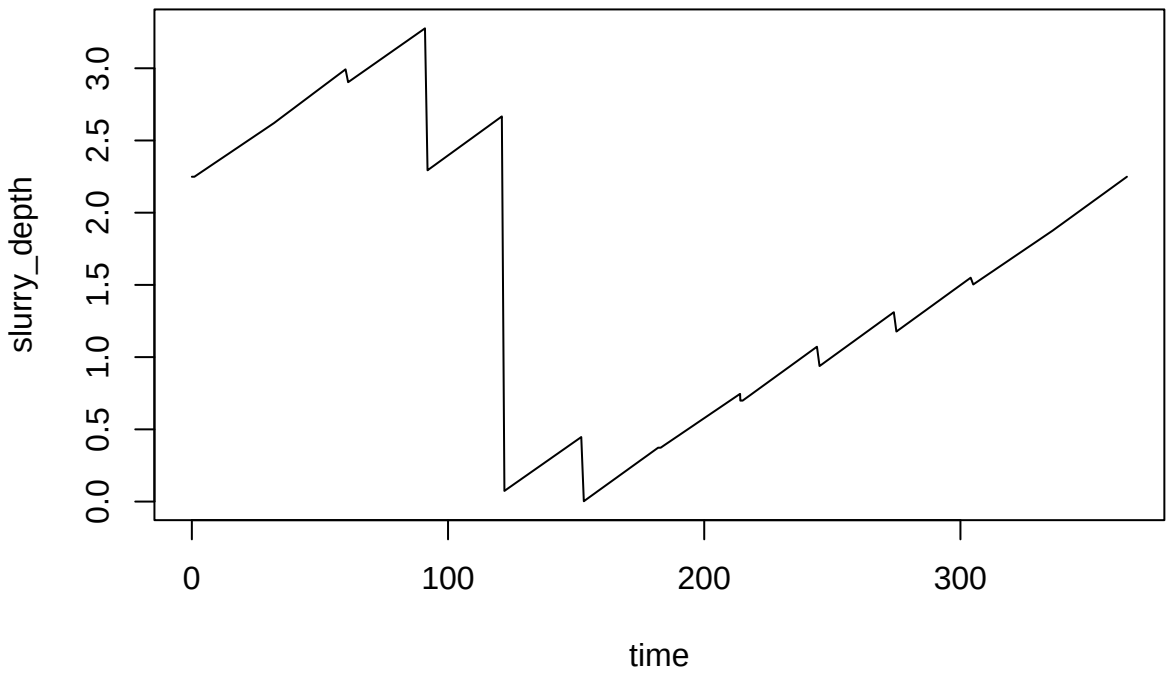
```
##
## Startup run 1x -> 2x -> and final run
```

```
##    user  system elapsed
##   1.233   0.000   1.238
```

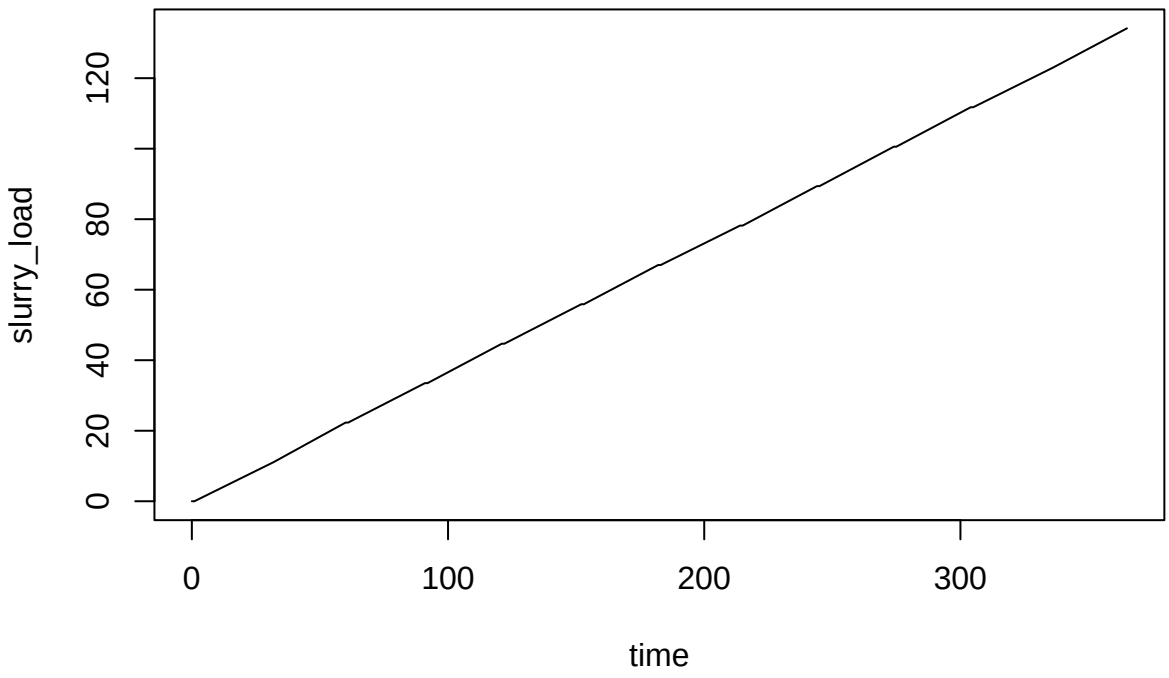
```
plot(slurry_mass ~ time, data = neo_out, type = 'l')
lines(slurry_mass ~ time, data = mass_series, type = 'l', col = 'red')
```



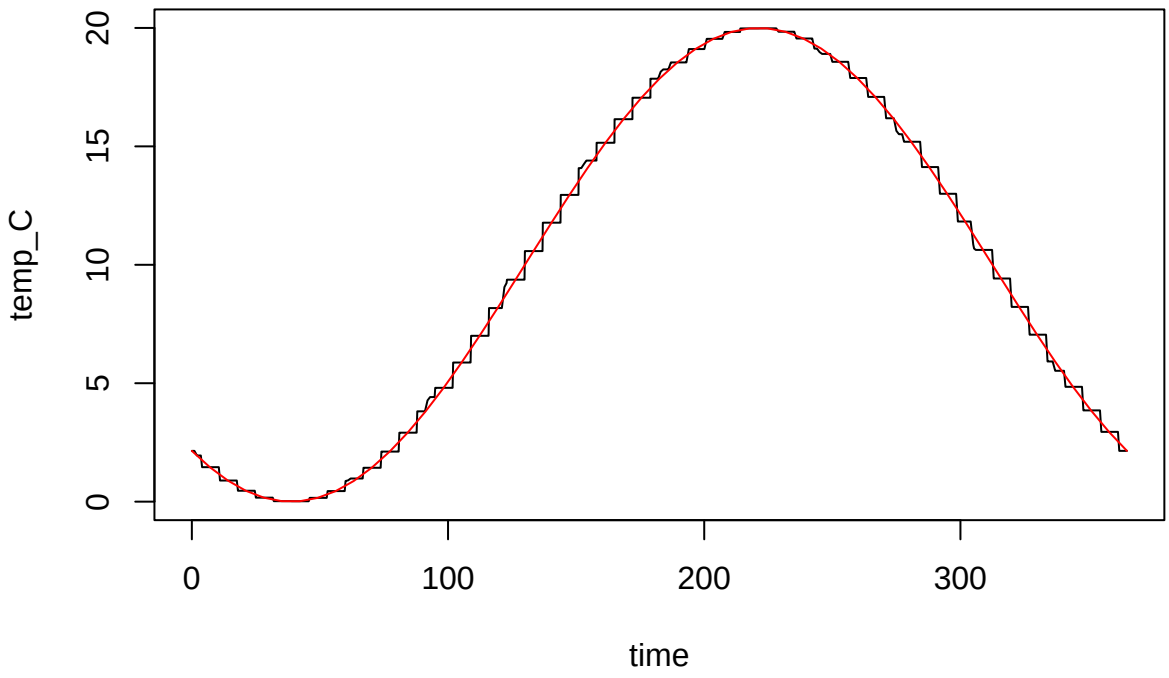

```
plot(slurry_depth ~ time, data = neo_out, type = 'l')
```



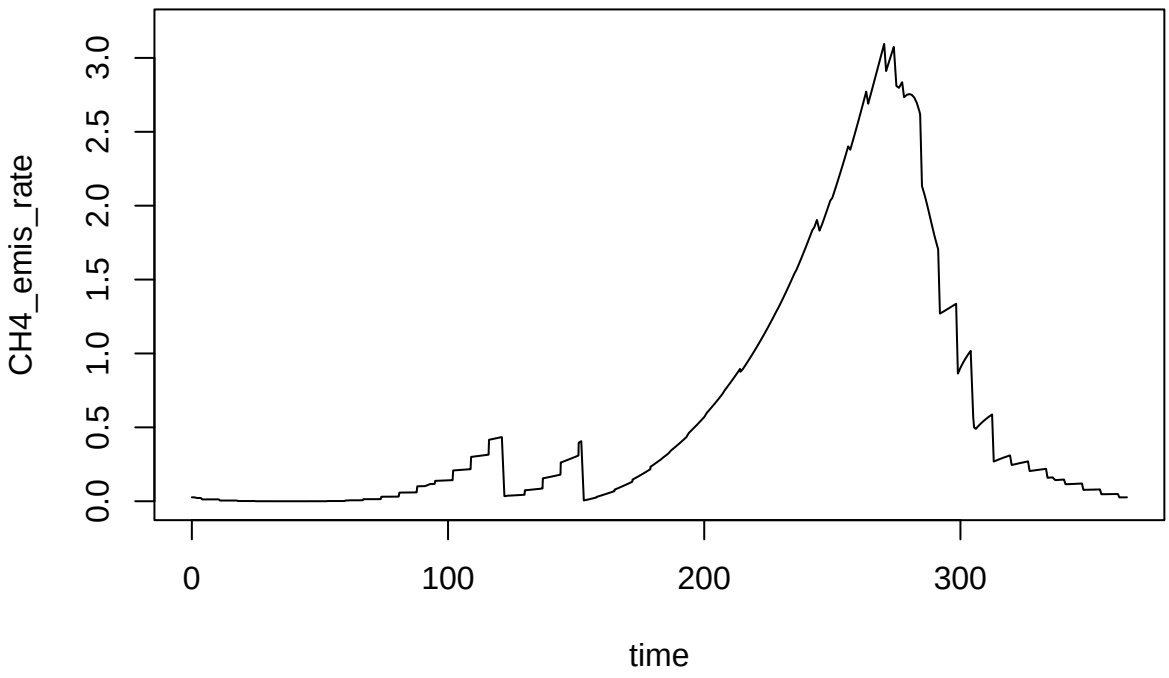

```
plot(slurry_load ~ time, data = neo_out, type = 'l')
```



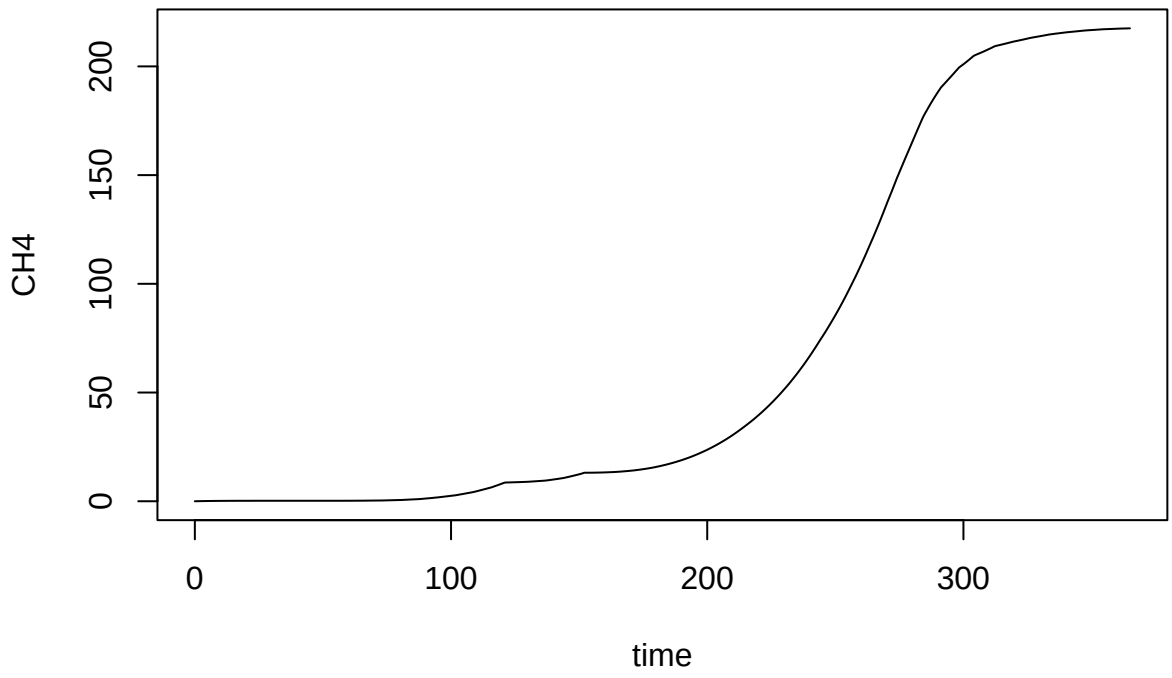

```
plot(temp_C ~ time, data = neo_out, type = 'l')
lines(temp_C ~ time, data = temp_dat, col = 'red')
```



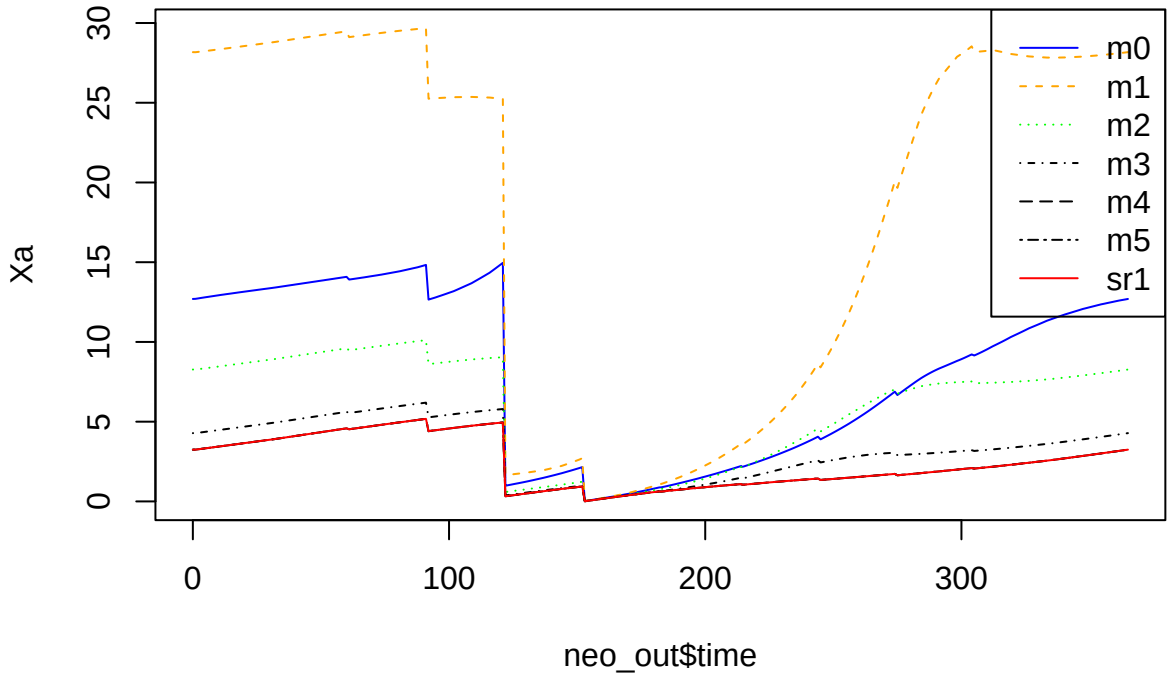

```
plot(CH4_emis_rate ~ time, data = neo_out, type = 'l', ylim = c(0, 3.2))
```



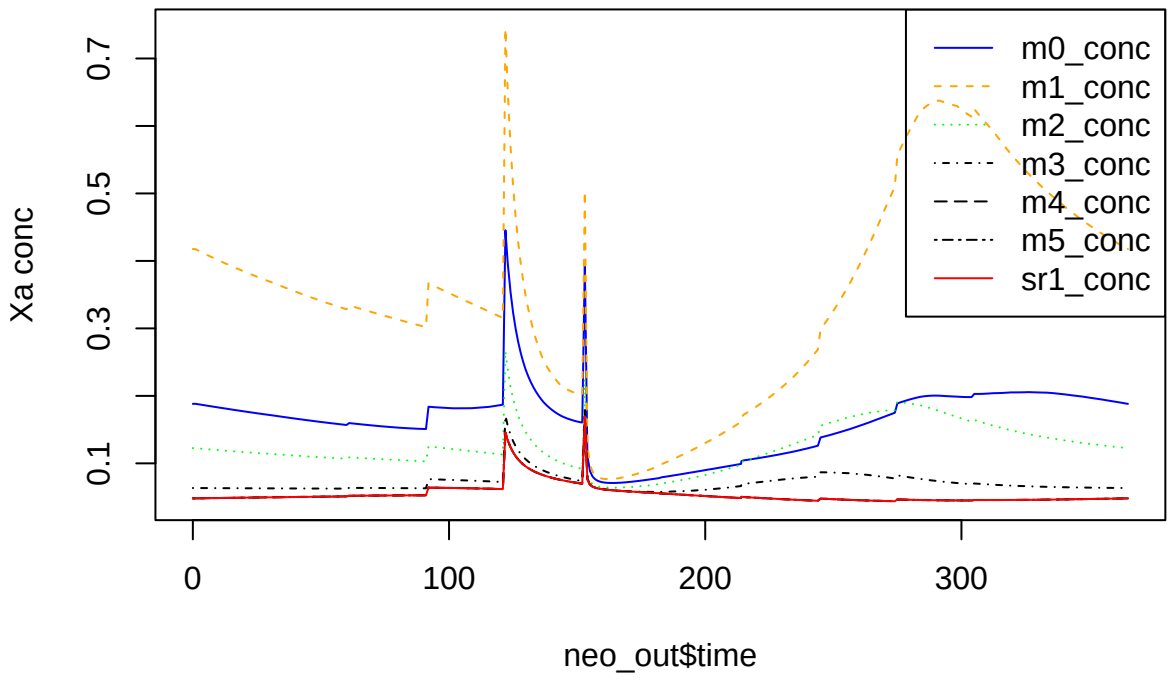

```
plot(CH4 ~ time, data = neo_out, type = 'l')
```



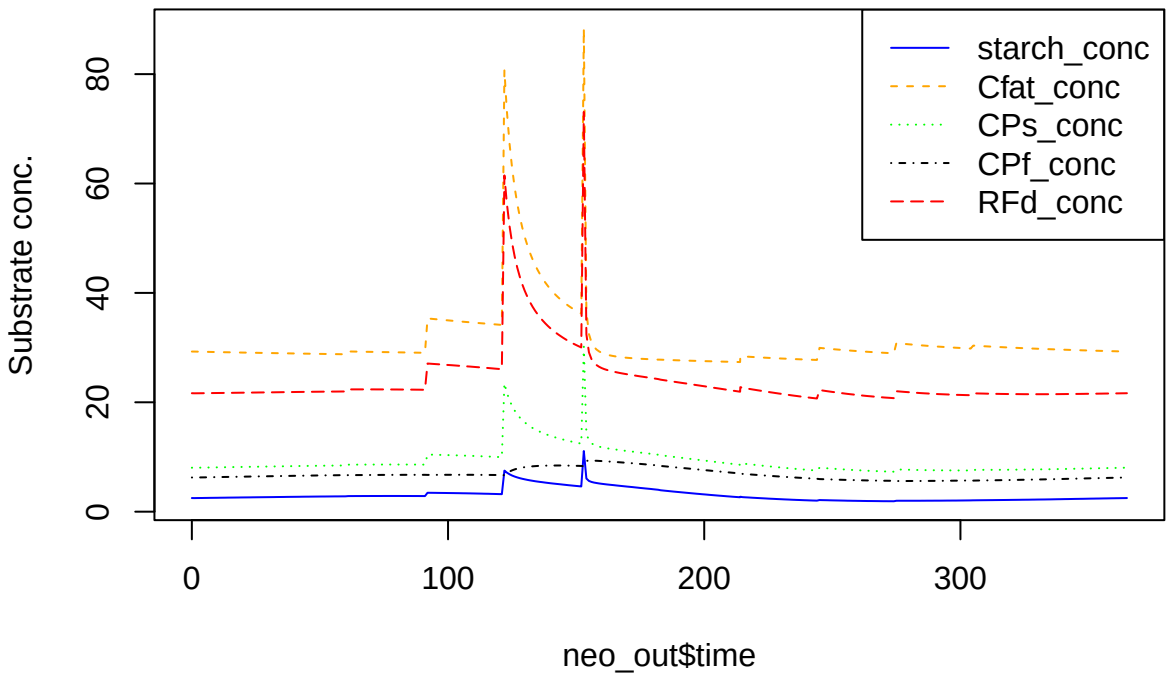

```
matplot(
  neo_out$time,
  neo_out[, nn <- grp_pars$grps],
  col = cc <- c('blue', 'orange', 'green', 'black', 'black', 'black', 'red'),
  lty = 1:length(nn), type = 'l',
  ylab = 'Xa'
)
legend('topright', legend = nn, col = cc, lty = 1:length(nn))
```



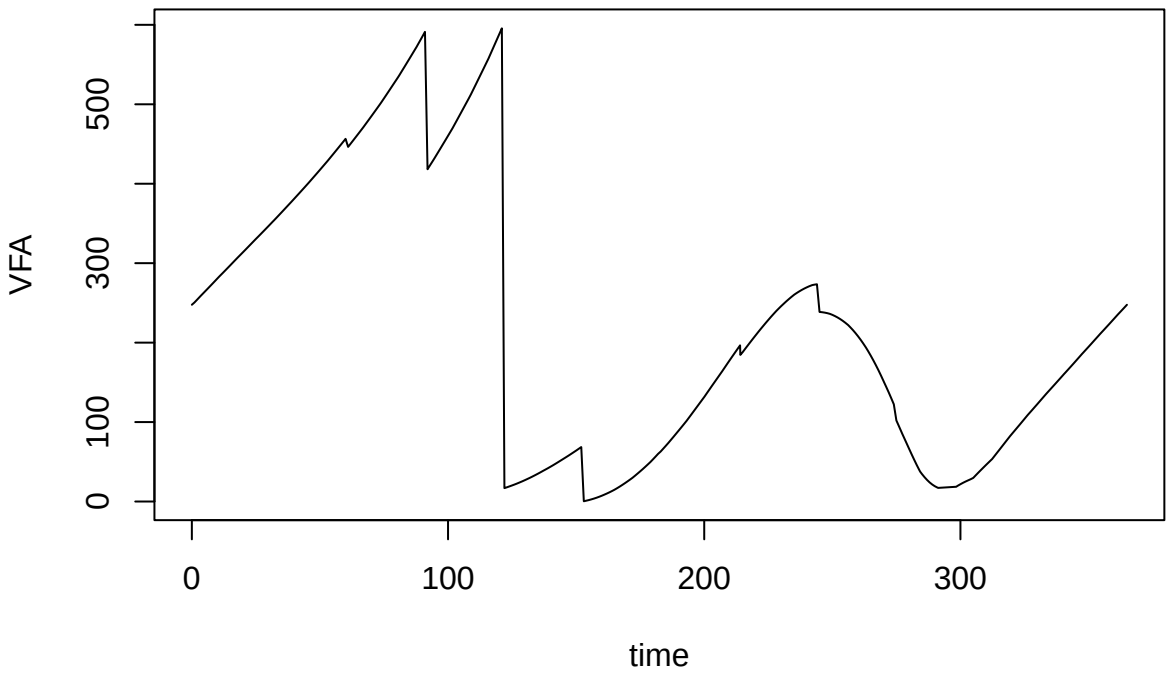

```
matplot(
  neo_out$time,
  neo_out[, nn <- paste0(grp_pars$grps, '_conc')],
  col = cc <- c('blue', 'orange', 'green', 'black', 'black', 'black', 'red'),
  lty = 1:length(nn),
  type = 'l',
  ylab = 'Xa conc'
)
legend('topright', legend = nn, col = cc, lty = 1:length(nn))
```



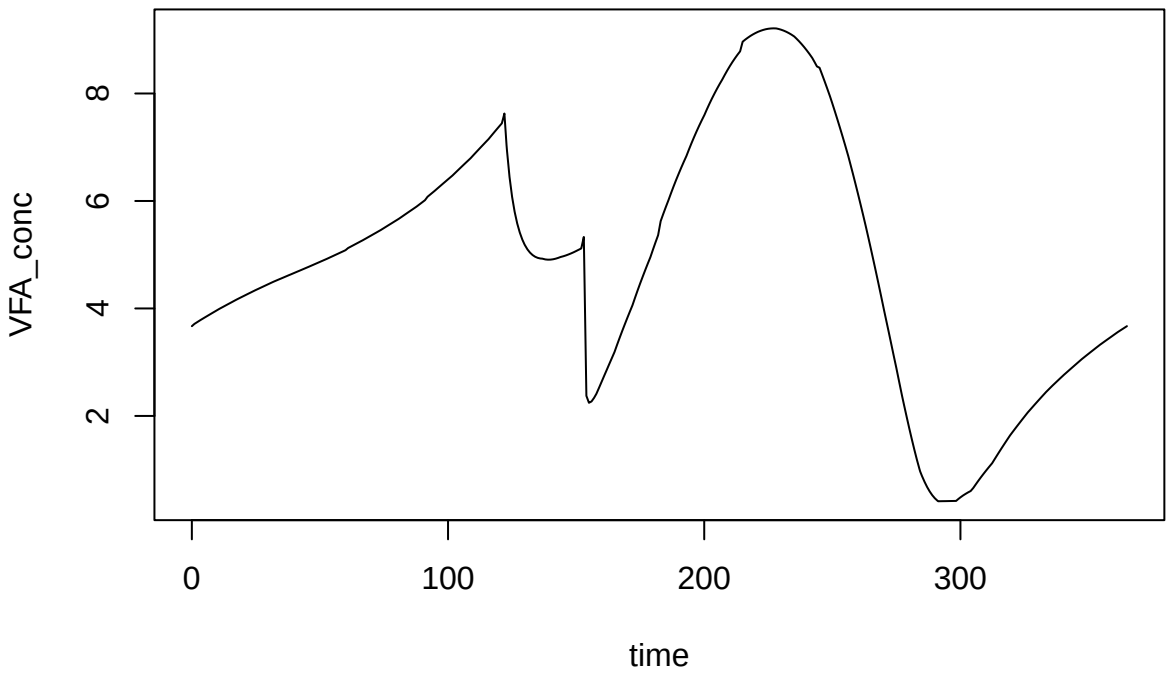

```
matplot(
  neo_out$time,
  neo_out[, nn <- paste0(sub_pars$subs, '_conc')],
  col = cc <- c('blue', 'orange', 'green', 'black', 'red'),
  lty = 1:length(nn),
  type = 'l',
  ylab = 'Substrate conc.'
)
legend('topright', legend = nn, col = cc, lty = 1:length(nn))
```




```
plot(VFA ~ time, data = neo_out, type = 'l')
```




```
plot(VFA_conc ~ time, data = neo_out, type = 'l')
```



No methanogens for comparison

```
grp_pars0 <- list(
  grps = c('m0', 'm1', 'm2','m3','m4','m5','sr1'),
  yield = c(default = 0.05, sr1 = 0.065),
  xa_fresh = c(default = 0),
  xa_init = c(default = 0),
  d_max = c(default = 0.02),
  qhat_opt =  c(m0 = 0.46289, m1 = 0.74568, m2 = 0.9006605, m3 = 2.79672, m4 = 4.66012, m5 = 7.4601, sr
  T_opt = c(m0 = 18, m1 = 18, m2 = 28, m3 = 36, m4 = 43.75, m5 = 55, sr1 = 43.75),
  T_min = c(m0 = 0, m1 = 9.67, m2 = 9.67, m3 = 15, m4 = 26.25, m5 = 30, sr1 = 0),
  T_max = c(m0 = 25, m1 = 25, m2 = 38, m3 = 45, m4 = 51.25, m5 = 60, sr1 = 51.25),
  ksmat = ksmat,
  mstoich = mstoich
)

grp_pars
```

```
## $grps
## [1] "m0"  "m1"  "m2"  "m3"  "m4"  "m5"  "sr1"
##
## $yield
## default     sr1
##   0.050   0.065
##
```

```
## $xa_fresh
##   m0   m1   m2   m3   m4   m5  sr1
## 0.06 0.06 0.06 0.06 0.06 0.06 0.06
##
## $xa_init
## default
##     0.1
##
## $d_max
## default
##    0.02
##
## $qhat_opt
##        m0        m1        m2        m3        m4        m5       sr1
## 0.4628900 0.7456800 0.9006605 2.7967200 4.6601200 7.4601000 0.0000000
##
## $T_opt
##    m0    m1    m2    m3    m4    m5   sr1
## 18.00 18.00 28.00 36.00 43.75 55.00 43.75
##
## $T_min
##    m0    m1    m2    m3    m4    m5   sr1
##  0.00  9.67  9.67 15.00 26.25 30.00  0.00
##
## $T_max
##    m0    m1    m2    m3    m4    m5   sr1
## 25.00 25.00 38.00 45.00 51.25 60.00 51.25
##
## $ksmat
##       m0   m1   m2   m3   m4   m5      sr1
## VFA 1.15 1.15 1.15 1.15 1.15 1.15 0.46000
## SO4   NA   NA   NA   NA   NA   NA 0.00694
##
## $mstoich
##     m0 m1 m2 m3 m4 m5      sr1
## VFA -1 -1 -1 -1 -1 -1 -1.00000
## SO4  0  0  0  0  0  0 -0.50156
## CH4  1  1  1  1  1  1  0.00000
```

```
devtools::load_all()
```

```
## i Loading abmneo
```

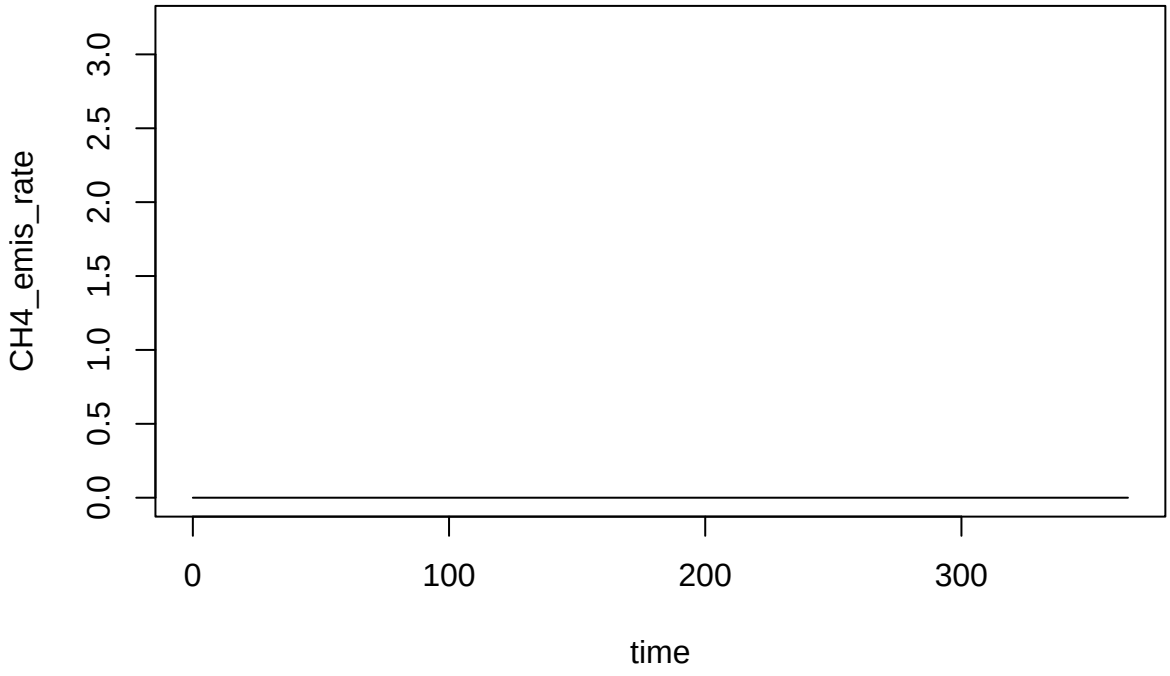
```
system.time(
  neo_out0 <- abmneo(
    storage = stor_ser,
    365,
    delta_t = 1,
    inf_pars = inf_pars,
    grp_pars = grp_pars0,
    sub_pars = sub_pars,
    chem_pars = chem_pars,
    var_pars = list(var = temp_dat),
    startup = 2
  )
```

```
)
```

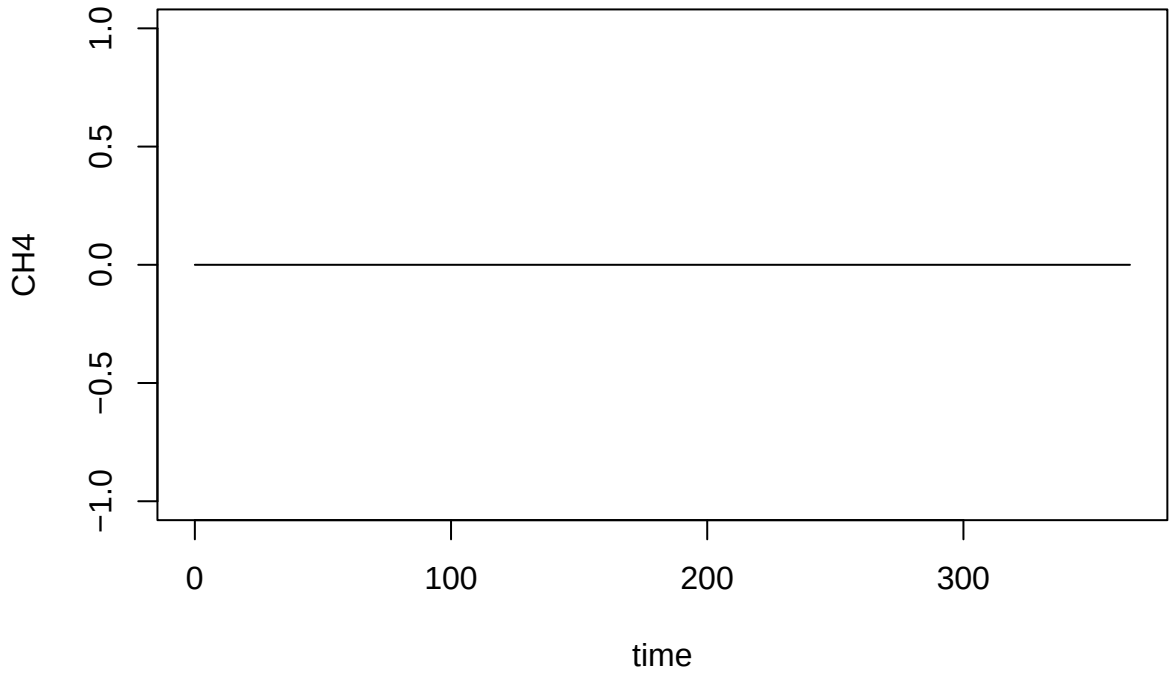
```
##
## Startup run 1x -> 2x -> and final run
```

```
##    user  system elapsed
##   1.202   0.001   1.208
```

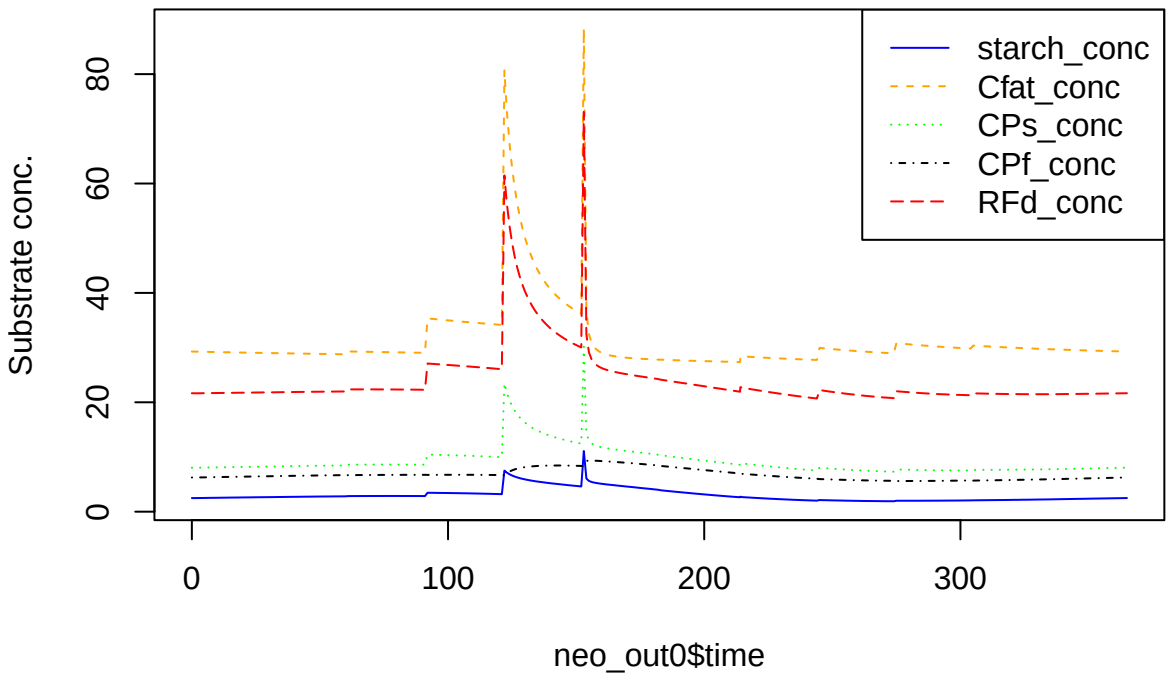
```
plot(CH4_emis_rate ~ time, data = neo_out0, type = 'l', ylim = c(0, 3.2))
```



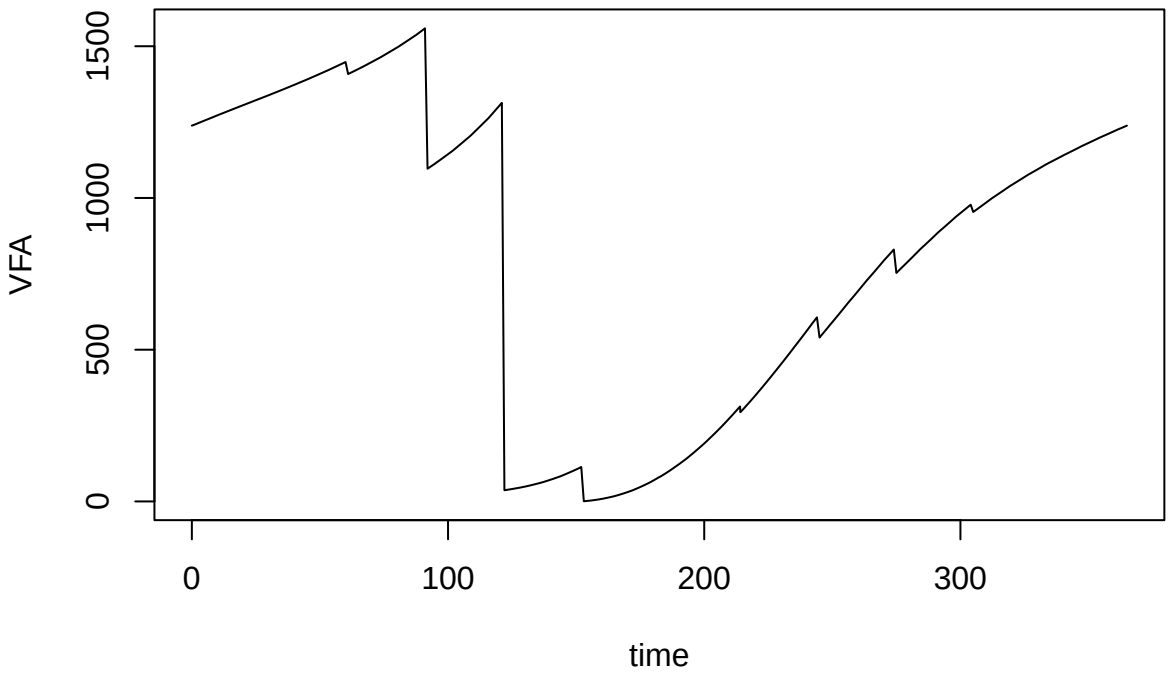

```
plot(CH4 ~ time, data = neo_out0, type = 'l')
```



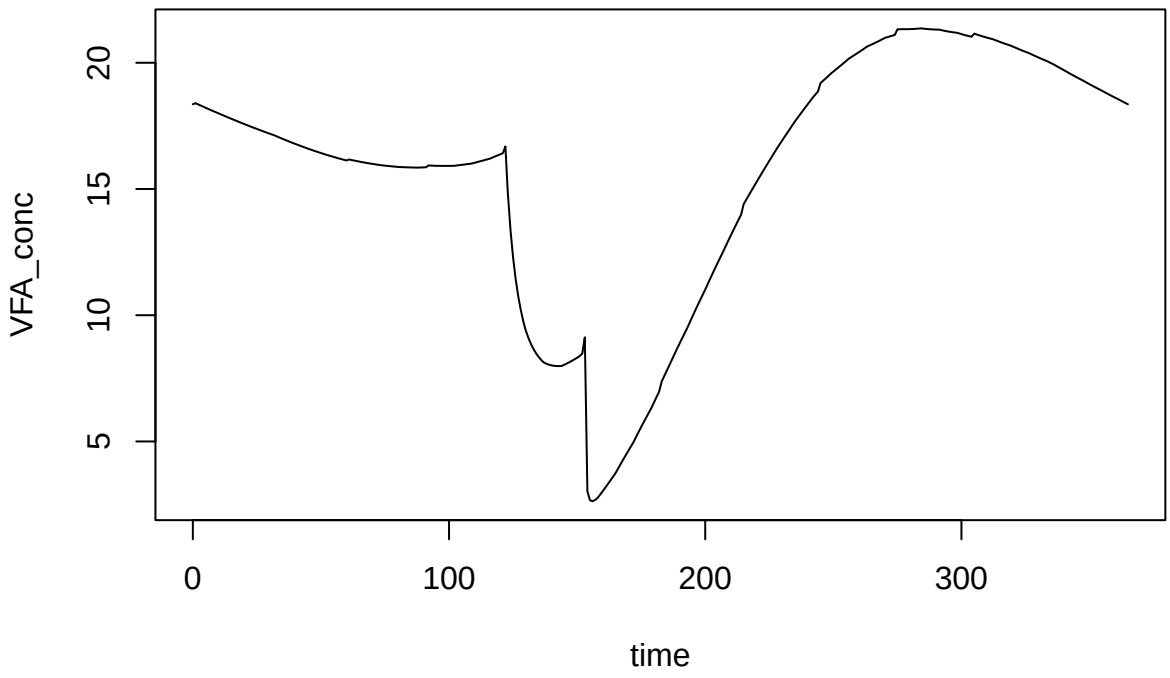

```
matplot(
  neo_out0$time,
  neo_out0[, nn <- paste0(sub_pars$subs, '_conc')],
  col = cc <- c('blue', 'orange', 'green', 'black', 'red'),
  lty = 1:length(nn),
  type = 'l',
  ylab = 'Substrate conc.'
)
legend('topright', legend = nn, col = cc, lty = 1:length(nn))
```




```
plot(VFA ~ time, data = neo_out0, type = 'l')
```




```
plot(VFA_conc ~ time, data = neo_out0, type = 'l')
```



Individual substrates for comparison

```
sub_pars_starch <- list(
  subs = c('starch', 'Cfat', 'CPs', 'CPf', 'RFd'),
  sub_enrich = c('starch', 'Cfat', 'CPs', 'RFd'),
  xd = 'xd',
  arrA = c(starch = 0.557 * 5 * 3.61e12, Cfat = 0.1 * 0.557 * 3.61e12, CPs = 0.557 * 2.065 * 3.61e12, CP
  arrE = c(default = 8.16e4),
  sub_fresh = c(starch = 1, Cfat = 0, CPs = 0, CPf = 0, RFd = 0, xd = 0),
  sub_init = c(starch = 1, Cfat = 0, CPs = 0, CPf = 0, RFd = 1, xd = 0)
)
```

```
neo_out_starch <- abmneo(
  storage = stor_ser,
  100,
  delta_t = 1,
  inf_pars = inf_pars,
  grp_pars = grp_pars0,
  sub_pars = sub_pars_starch,
  chem_pars = chem_pars,
  var_pars = list(var = temp_dat),
  add_pars = list(VFA_fresh = 0, VFA_init = 0),
  startup = 0
)
```

Numeric output

```
neo_out_starch$sum <- neo_out_starch$starch_conc + neo_out_starch$VFA_conc
neo_out_starch[1:40, c('time', 'starch_conc', 'VFA_conc', 'sum')]
```

```
##         time starch_conc    VFA_conc      sum
## 1   0.000000   1.0000000 0.000000000 1.000000
## 2   1.000000   0.9967028 0.003957453 1.000660
## 3   2.000000   0.9935248 0.007771902 1.001297
## 4   3.000000   0.9903735 0.011552496 1.001926
## 5   3.509615   0.9887777 0.013466288 1.002244
## 6   4.000000   0.9873450 0.015184085 1.002529
## 7   5.000000   0.9844411 0.018664634 1.003106
## 8   6.000000   0.9815605 0.022115483 1.003676
## 9   7.000000   0.9787030 0.025537071 1.004240
## 10  8.000000   0.9758681 0.028929829 1.004798
## 11  9.000000   0.9730556 0.032294176 1.005350
## 12 10.000000   0.9702652 0.035630527 1.005896
## 13 10.528846   0.9687983 0.037383763 1.006182
## 14 11.000000   0.9675940 0.038822761 1.006417
## 15 12.000000   0.9650520 0.041858963 1.006911
## 16 13.000000   0.9625290 0.044870983 1.007400
## 17 14.000000   0.9600247 0.047859163 1.007884
## 18 15.000000   0.9575387 0.050823843 1.008363
## 19 16.000000   0.9550710 0.053765349 1.008836
## 20 17.000000   0.9526212 0.056684005 1.009305
## 21 17.548077   0.9512860 0.058274073 1.009560
## 22 18.000000   0.9502550 0.059501477 1.009757
## 23 19.000000   0.9479854 0.062202626 1.010188
## 24 20.000000   0.9457314 0.064883604 1.010615
## 25 21.000000   0.9434930 0.067544686 1.011038
## 26 22.000000   0.9412700 0.070186143 1.011456
## 27 23.000000   0.9390621 0.072808237 1.011870
## 28 24.000000   0.9368691 0.075411228 1.012280
## 29 24.567308   0.9356315 0.076879530 1.012511
## 30 25.000000   0.9347311 0.077947559 1.012679
## 31 26.000000   0.9326598 0.080403369 1.013063
## 32 27.000000   0.9306020 0.082841884 1.013444
## 33 28.000000   0.9285575 0.085263332 1.013821
## 34 29.000000   0.9265261 0.087667934 1.014194
## 35 30.000000   0.9245077 0.090055908 1.014564
## 36 31.000000   0.9225021 0.092427469 1.014930
## 37 31.586538   0.9213316 0.093810928 1.015143
## 38 32.000000   0.9205279 0.094760620 1.015289
## 39 33.000000   0.9186340 0.096997042 1.015631
## 40 34.000000   0.9167523 0.099217614 1.015970
```